EC

EL CURIO | RETAILERP

Deployment Guide

Docker & IIS Deployment — Step-by-Step Reference

Docker Compose | IIS on Windows Server | .NET 8 / ASP.NET Core | Next.js 15 | SQL Server 2022 | Redis 7

Version	Date	Platform	Audience
v2.1	March 2026	Windows / Linux	DevOps / SysAdmin

Table of Contents

1 Architecture Overview 3

System Components & Service Map 3

Port Reference 3

Default Credentials 4

2 Docker Deployment 5

Prerequisites 5

Quick Start — Docker Compose 5

Environment Configuration (.env) 6

Building Individual Service Images 7

Health Checks & Service Dependencies 7

Common Docker Operations 8

Volumes & Data Persistence 8

Production Hardening for Docker 9

3 IIS Deployment (Windows Server) 10

Prerequisites & Windows Features 10

Step 1 — SQL Server Setup 11

Step 2 — Redis Setup 11

Step 3 — Publish Microservices 12

Step 4 — IIS Application Pools & Sites 13

Step 5 — web.config for each Service 14

Step 6 — Deploy API Gateway 15

Step 7 — Deploy Next.js Frontend 16

Step 8 — IIS Reverse Proxy (ARR) 17

Step 9 — SSL/TLS Configuration 18

Windows Service Alternative 19

4 Configuration Reference 20

appsettings.json per Environment 20

Frontend Environment Variables 20

JWT & Security Settings 21

5 Troubleshooting 22
Docker Common Issues 22
IIS Common Issues 22
Database Connectivity Issues 23

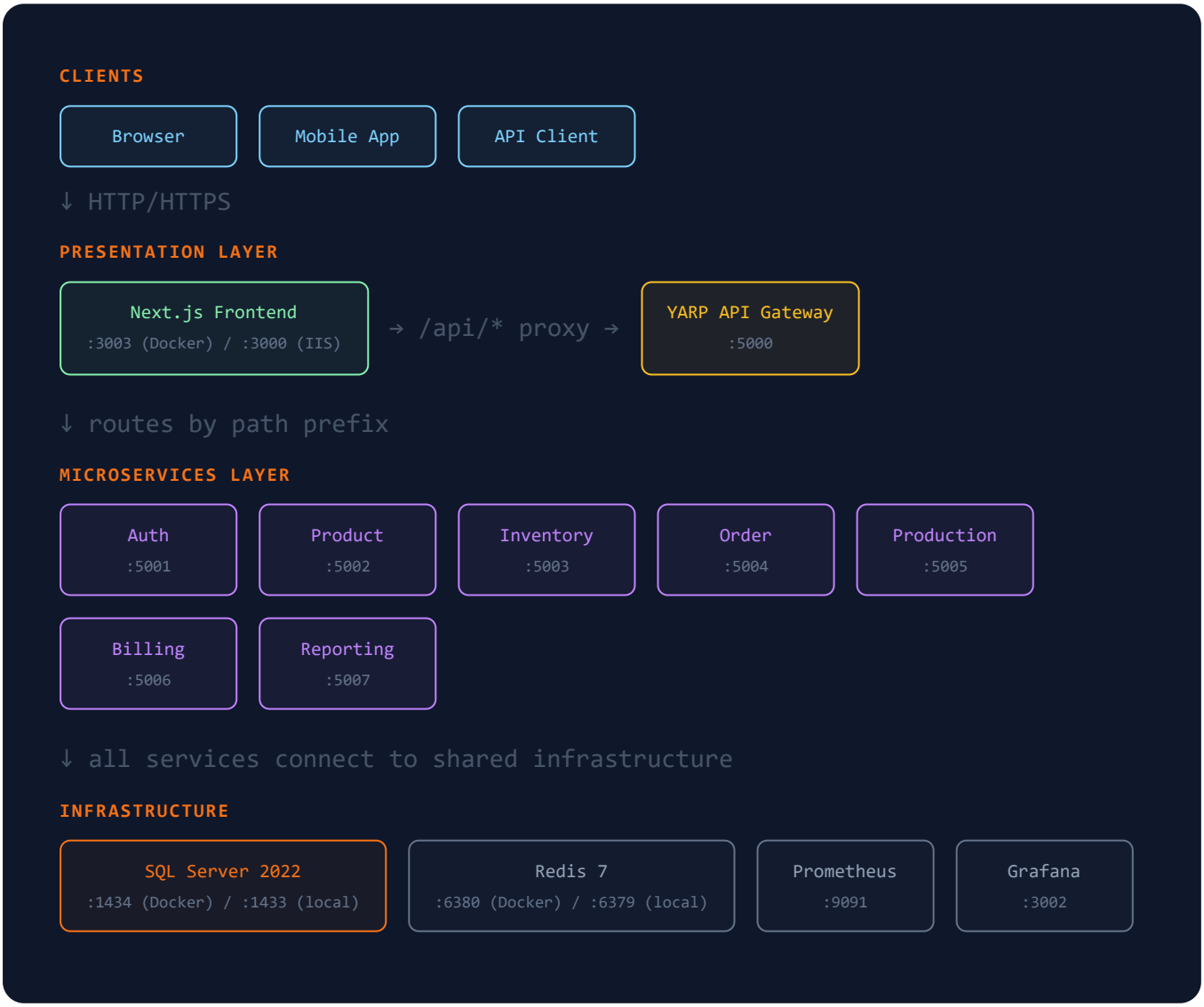
CHAPTER 1

Architecture Overview

System components, service topology, and port assignments

System Architecture

RetailERP is a microservices-based platform. All client requests flow through the YARP API Gateway. The Next.js frontend proxies API calls to the gateway. Each backend service is an independent ASP.NET Core application with its own database schema.



Port Reference

3003
Frontend (Docker)
Next.js web application

5000
API Gateway
YARP reverse proxy, all API traffic

5001
Auth Service
Login, JWT, user management

5002
Product Service
Articles, SKUs, masters

5003
Inventory Service
Stock, GRN, dispatch, freeze

5004
Order Service
Customer orders, channels

5005
Production Service
Production orders, job work

5006
Billing Service
Invoices, packing, delivery

5007
Reporting Service
Analytics and reports

1434
SQL Server
Docker mapped; local uses 1433

6380
Redis
Docker mapped; local uses 6379

3002
Grafana
Metrics dashboard

Default Credentials

Service	Username / User	Password	Notes
RetailERP Application	`admin@elcurio.com`	`Admin@123`	Change after first login
SQL Server (SA)	`sa`	`RetailERP@2024!`	Set in `.env` / docker-compose
Grafana	`admin`	`admin`	Set `GF_SECURITY_ADMIN_PASSWORD`
Redis	—	—	No password in default config

Change All Credentials Before Production

The passwords shown above are development defaults. Before deploying to production, change the SA password, the JWT secret (minimum 32 characters), the Grafana admin password, and the application admin account password. Store secrets in environment variables or a secrets manager — never commit them in source control.

CHAPTER 2

Docker Deployment

Deploy all services using Docker Compose — recommended for staging and production on Linux/Windows

Prerequisites

Requirement	Minimum Version	Notes
Docker Engine	24.0+	Linux, Windows (WSL2), or Docker Desktop
Docker Compose	v2.20+ (plugin)	`docker compose` (not legacy `docker-compose`)
Available RAM	8 GB minimum	16 GB recommended for all services
Available Disk	20 GB free	For images, DB volumes, and logs
CPU	4 cores	8+ cores recommended
Git	Any recent version	To clone the repository

Quick Start — Docker Compose

Step 1 — Clone & configure environment

```bash
# Clone the repository
git clone https://github.com/your-org/RetailERP.git
cd RetailERP

# Copy and edit the environment file
cp .env.example .env
nano .env    # set SA_PASSWORD and JWT_SECRET
```

Step 2 — Build and start all services

```bash
# Build all images and start in detached mode
bash scripts/deploy-docker.sh up

# OR using Docker Compose directly
docker compose build --parallel
docker compose up -d
```

Step 3 — Verify all services are running

```bash
docker compose ps

# Expected output — all services should show "healthy" or "running":
# NAME                    STATUS          PORTS
# retailerp-sqlserver     healthy         0.0.0.0:1434->1433/tcp
# retailerp-redis         healthy         0.0.0.0:6380->6379/tcp
# retailerp-auth          healthy         0.0.0.0:5001->8080/tcp
# retailerp-product       running         0.0.0.0:5002->8080/tcp
# retailerp-inventory     running         0.0.0.0:5003->8080/tcp
# retailerp-gateway       healthy         0.0.0.0:5000->8080/tcp
# retailerp-frontend      running         0.0.0.0:3003->3000/tcp
```

Step 4 — Run database initialization

The `db-init` container runs automatically on first startup and executes all SQL scripts from the `database/` folder in order: schemas → tables → indexes → stored-procedures → seed-data. This container exits once complete.

```bash
# Watch initialization logs
docker logs retailerp-db-init --follow

# Expected last line:
# Database initialization complete!
```

Application is Ready

Once all services are healthy, open **http://localhost:3003** in your browser and log in with **admin@elcurio.com / Admin@123**.

Environment Configuration (.env)

The root `.env` file provides secrets to Docker Compose. Never commit this file to source control.

```env
# SQL Server SA password — must match across all services
SA_PASSWORD=RetailERP@2024!

# JWT signing secret — minimum 32 characters, keep secret
JWT_SECRET=RetailERP-SuperSecret-JWT-Key-2024-Must-Be-At-Least-32-Characters

# Grafana admin password
GRAFANA_PASSWORD=admin
```

These values are injected into each container via the `x-common-env` YAML anchor in `docker-compose.yml`:

```yaml
x-common-env: &common-env
  ASPNETCORE_ENVIRONMENT: "Docker"
  ASPNETCORE_URLS:         "http://+:8080"
  ConnectionStrings__DefaultConnection: "Server=sqlserver;Database=RetailERP;User Id=sa;Password=RetailERP
@2024!;TrustServerCertificate=true"
  Jwt__Secret:    "RetailERP-SuperSecret-JWT-Key-2024..."
  Jwt__Issuer:    "RetailERP.Auth"
  Jwt__Audience:  "RetailERP.Client"
```

docker-compose.yml (excerpt)

Building Individual Service Images

Each microservice uses the same parameterized `Dockerfile.api` with build arguments `SERVICE_NAME` and `SERVICE_DIR`.

```bash
# Build the Inventory service image
docker build \
  -f docker/Dockerfile.api \
  --build-arg SERVICE_NAME=Inventory \
  --build-arg SERVICE_DIR=inventory \
  -t retailerp-inventory:latest \
  .

# Build the Frontend image
docker build \
  -f docker/Dockerfile.frontend \
  -t retailerp-frontend:latest \
  .

# Build the Gateway image
docker build \
  -f docker/Dockerfile.gateway \
  -t retailerp-gateway:latest \
  .
```

bash — build individual service

Health Checks & Service Dependencies

Services start in dependency order enforced by Docker Compose health checks:

Service	Waits For	Health Check	Interval
db-init	sqlserver (healthy)	SQL SELECT 1	15s
auth-api	sqlserver, redis (healthy)	`curl /health`	15s / 5 retries
product-, inventory-, order-, billing-api	sqlserver (healthy)	`curl /health`	30s / 3 retries
gateway	auth-api (healthy)	`curl /health`	30s / 3 retries
frontend	gateway (started)	`wget localhost:3000`	30s / 3 retries

Common Docker Operations

```bash
# View logs for all services (live)
docker compose logs -f

# View logs for a specific service
docker compose logs -f inventory-api

# Restart a single service
docker compose restart inventory-api

# Rebuild and redeploy a single service (zero-downtime not guaranteed)
docker compose build inventory-api
docker compose up -d --no-deps inventory-api

# Stop all services (preserves volumes)
docker compose down

# Stop and remove all data volumes (DESTRUCTIVE — deletes database)
docker compose down -v

# Check resource usage
docker stats

# Execute a command inside a running container
docker exec -it retailerp-sqlserver /bin/bash

# Scale a stateless service (only for stateless services)
docker compose up -d --scale product-api=2
```

Volumes & Data Persistence

The following Docker named volumes persist data between container restarts:

Volume Name	Service	Contents
`sqlserver-data`	sqlserver	All database files (`/var/opt/mssql`)
`redis-data`	redis	Redis persistence snapshots (`/data`)
`prometheus-data`	prometheus	Metrics time-series data
`grafana-data`	grafana	Dashboard definitions and settings

bash — backup SQL Server data

```bash
# Backup the SQL Server database to a local file
docker exec retailerp-sqlserver \
  /opt/mssql-tools18/bin/sqlcmd \
  -S localhost -U sa -P 'RetailERP@2024!' -C \
  -Q "BACKUP DATABASE RetailERP TO DISK='/var/opt/mssql/backup/RetailERP.bak' WITH INIT"

# Copy backup from container to host
docker cp retailerp-sqlserver:/var/opt/mssql/backup/RetailERP.bak ./RetailERP.bak
```

Production Hardening for Docker

- Use **Docker secrets** or an external secrets manager (HashiCorp Vault, AWS Secrets Manager) instead of plain `.env` files.
- Replace the shared SA password with a dedicated application database user with minimal privileges.
- Add a reverse proxy (Nginx, Traefik, or Caddy) in front of the frontend and gateway containers to handle SSL termination.
- Set `ASPNETCORE_ENVIRONMENT=Production` in all microservice containers (disables Swagger UI and developer error pages).
- Enable Redis password authentication and set `--requirepass` in the Redis command.
- Configure log drivers (`json-file` with rotation, or forward to a centralized log system) to prevent disk fill.
- Use container resource limits (`mem_limit` , `cpus`) to prevent any one service from exhausting host resources.
- Generate a strong JWT secret: `openssl rand -base64 48` .

docker-compose.override.yml — production limits example

```yaml
services:
  inventory-api:
    deploy:
      resources:
        limits:
          cpus: '1.0'
          memory: 512M
        reservations:
          memory: 256M
    logging:
      driver: "json-file"
      options:
        max-size: "50m"
        max-file: "5"
```

CHAPTER 3

IIS Deployment (Windows Server)

Host all services on Internet Information Services — suited for Windows Server environments without Docker

Prerequisites & Windows Features

Requirement	Version / Details	How to Install
Windows Server	2019 or 2022	—
IIS	10.0	Server Manager → Add Roles → Web Server (IIS)
.NET Hosting Bundle	8.0 (includes ASP.NET Core Runtime + IIS module)	Download from `dotnet.microsoft.com/download`
Node.js	20 LTS	For running Next.js standalone server
SQL Server	2019 or 2022	Standard or Express edition
Redis for Windows	7.x (Memurai or WSL2)	Memurai: `memurai.com`
ARR + URL Rewrite	IIS modules	Web Platform Installer or manual MSI
.NET SDK 8.0	For publishing	On the build machine

Enable required IIS features (PowerShell — run as Administrator)

PowerShell — run as Administrator

```powershell
# Enable IIS with common features
Enable-WindowsOptionalFeature -Online -FeatureName `
  IIS-WebServerRole, IIS-WebServer, IIS-CommonHttpFeatures,
  IIS-HttpErrors, IIS-ApplicationDevelopment,
  IIS-NetFxExtensibility45, IIS-ASPNET45,
  IIS-ISAPIExtensions, IIS-ISAPIFilter,
  IIS-DefaultDocument, IIS-DirectoryBrowsing,
  IIS-StaticContent, IIS-HttpCompressionStatic,
  IIS-RequestFiltering, IIS-HttpLogging,
  IIS-ManagementConsole -All

# Verify IIS is running
Get-Service W3SVC
```

Step 1 — SQL Server Setup

1. Install SQL Server 2022 (Standard or Developer edition).
2. Enable **SQL Server Authentication** mode (Mixed Mode) and set the `sa` password to `RetailERP@2024!` (or your chosen password).
3. Create the database and run all initialization scripts from the `database/` folder in order:

PowerShell — run SQL scripts

```
$server   = "localhost"
$user     = "sa"
$password = "RetailERP@2024!"
$dbname   = "RetailERP"
$sqlcmd   = "sqlcmd"

# Create database
& $sqlcmd -S $server -U $user -P $password `
  -Q "IF NOT EXISTS (SELECT * FROM sys.databases WHERE name='$dbname') CREATE DATABASE $dbname"

# Run all scripts in order
$scriptFolders = @('schemas', 'tables', 'indexes', 'stored-procedures', 'seed-data')
foreach ($folder in $scriptFolders) {
  Get-ChildItem ".\database\$folder\*.sql" | ForEach-Object {
    Write-Host "Running $_"
    & $sqlcmd -S $server -U $user -P $password -d $dbname -i $_.FullName
  }
}
```

Step 2 — Redis Setup

For Windows Server, use **Memurai** (Redis-compatible, native Windows service) or run Redis via WSL2.

PowerShell — install Memurai as Windows Service

```
# Download Memurai from memurai.com/get-memurai
# After installation, it registers as a Windows Service automatically

# Verify Redis is running
redis-cli ping
# Expected response: PONG

# Check service status
Get-Service Memurai
```

Step 3 — Publish Microservices

Publish each microservice as a self-contained deployment from the build machine (requires .NET SDK 8.0).

PowerShell — publish all services

```powershell
$services = @(
  @{ name='Auth';       dir='auth' },
  @{ name='Product';    dir='product' },
  @{ name='Inventory';  dir='inventory' },
  @{ name='Order';      dir='order' },
  @{ name='Production'; dir='production' },
  @{ name='Billing';    dir='billing' },
  @{ name='Reporting';  dir='reporting' },
  @{ name='Gateway';    dir='gateway' }
)

foreach ($svc in $services) {
  $proj = if ($svc.name -eq 'Gateway') {
    "src\services\gateway\RetailERP.Gateway.csproj"
  } else {
    "src\services\$($svc.dir)\RetailERP.$($svc.name).API\RetailERP.$($svc.name).API.csproj"
  }
  dotnet publish $proj `
    -c Release `
    -r win-x64 `
    --self-contained false `
    -o "C:\RetailERP\services\$($svc.name)"
  Write-Host "Published $($svc.name)"
}
```

Step 4 — IIS Application Pools & Sites

Create one Application Pool per service. All pools should use **No Managed Code** (ASP.NET Core runs out-of-process).

PowerShell — create app pools and sites

```powershell
Import-Module WebAdministration

$services = @(
  @{ name='Auth';       port=5001 },
  @{ name='Product';    port=5002 },
  @{ name='Inventory';  port=5003 },
  @{ name='Order';      port=5004 },
  @{ name='Production'; port=5005 },
  @{ name='Billing';    port=5006 },
  @{ name='Reporting';  port=5007 },
  @{ name='Gateway';    port=5000 }
)

foreach ($svc in $services) {
  $poolName = "RetailERP-$($svc.name)"
  $siteName = "RetailERP-$($svc.name)"
  $physPath = "C:\RetailERP\services\$($svc.name)"

  # Create App Pool (No Managed Code for ASP.NET Core)
  New-WebAppPool -Name $poolName
  Set-ItemProperty "IIS:\AppPools\$poolName" `
    -Name managedRuntimeVersion -Value ""

  # Create IIS Site
  New-Website -Name $siteName `
    -ApplicationPool $poolName `
    -PhysicalPath $physPath `
    -Port $svc.port `
    -Force
}
```

Step 5 — web.config for Each Microservice

Each published microservice needs a `web.config` in its publish directory. The ASP.NET Core module (ANCM) is installed by the .NET Hosting Bundle.

C:\RetailERP\services\Inventory\web.config

```xml
<?xml version="1.0" encoding="utf-8"?>
<configuration>
  <system.webServer>
    <handlers>
      <add name="aspNetCore"
           path="*"
           verb="*"
           modules="AspNetCoreModuleV2"
           resourceType="Unspecified" />
    </handlers>
    <aspNetCore
        processPath="dotnet"
        arguments=".\RetailERP.Inventory.API.dll"
        stdoutLogEnabled="true"
        stdoutLogFile=".\logs\stdout"
        hostingModel="inprocess">
      <environmentVariables>
        <environmentVariable name="ASPNETCORE_ENVIRONMENT" value="Production" />
        <environmentVariable name="ConnectionStrings__DefaultConnection"
                             value="Server=localhost;Database=RetailERP;User Id=retailerp_app;Password=App
Pass@2024!;TrustServerCertificate=true" />
        <environmentVariable name="Jwt__Secret"
                             value="YOUR-PRODUCTION-JWT-SECRET-HERE" />
        <environmentVariable name="Jwt__Issuer"    value="RetailERP.Auth" />
        <environmentVariable name="Jwt__Audience"  value="RetailERP.Client" />
      </environmentVariables>
    </aspNetCore>
  </system.webServer>
</configuration>
```

Repeat for Each Service

Create a similar `web.config` in each service's publish folder, adjusting the **arguments** value to match the DLL name: `RetailERP.Auth.API.dll`, `RetailERP.Product.API.dll`, `RetailERP.Gateway.dll`, etc. Create a `logs\` subfolder in each service directory for stdout logging.

Step 6 — Deploy API Gateway

The API Gateway (YARP) is an ASP.NET Core app deployed identically to the microservices. It listens on port 5000 and routes requests to each microservice on localhost.

C:\RetailERP\services\Gateway\web.config

```xml
<?xml version="1.0" encoding="utf-8"?>
<configuration>
  <system.webServer>
    <handlers>
      <add name="aspNetCore" path="*" verb="*"
           modules="AspNetCoreModuleV2" resourceType="Unspecified" />
    </handlers>
    <aspNetCore
        processPath="dotnet"
        arguments=".\RetailERP.Gateway.dll"
        stdoutLogEnabled="true"
        stdoutLogFile=".\logs\stdout"
        hostingModel="inprocess">
      <environmentVariables>
        <environmentVariable name="ASPNETCORE_ENVIRONMENT" value="Production" />
        <environmentVariable name="Cors__Origins__0" value="https://yourdomain.com" />
      </environmentVariables>
    </aspNetCore>
  </system.webServer>
</configuration>
```

Step 7 — Deploy Next.js Frontend

Next.js with `output: "standalone"` produces a `.next/standalone` folder that contains a `server.js` Node.js server. On IIS, this is hosted via the **iisnode** module which proxies to Node.js.

Build the frontend

PowerShell — build Next.js

```powershell
cd src\frontend

# Set production API URL
$env:NEXT_PUBLIC_API_URL = "http://localhost:5000"

npm ci
npm run build

# Copy standalone output to deployment folder
xcopy /E /I /Y .next\standalone C:\RetailERP\frontend
xcopy /E /I /Y .next\static     C:\RetailERP\frontend\.next\static
xcopy /E /I /Y public           C:\RetailERP\frontend\public
```

Install iisnode and create IIS site for frontend

PowerShell

```powershell
# Install iisnode (download from github.com/Azure/iisnode)
# Then create IIS site for frontend on port 3000
New-WebAppPool -Name "RetailERP-Frontend"
Set-ItemProperty "IIS:\AppPools\RetailERP-Frontend" `
  -Name managedRuntimeVersion -Value ""

New-Website -Name "RetailERP-Frontend" `
  -ApplicationPool "RetailERP-Frontend" `
  -PhysicalPath "C:\RetailERP\frontend" `
  -Port 3000
```

C:\RetailERP\frontend\web.config — iisnode configuration

```xml
<?xml version="1.0" encoding="utf-8"?>
<configuration>
  <system.webServer>
    <handlers>
      <add name="iisnode"
           path="server.js"
           verb="*"
           modules="iisnode" />
    </handlers>
    <rewrite>
      <rules>
        <rule name="Node" stopProcessing="true">
          <match url=".*" />
          <conditions logicalGrouping="MatchAll" trackAllCaptures="false">
            <add input="{REQUEST_FILENAME}" matchType="IsFile" negate="true" />
          </conditions>
          <action type="Rewrite" url="server.js" />
        </rule>
      </rules>
    </rewrite>
    <iisnode
      nodeProcessCommandLine="node"
      loggingEnabled="true"
      logDirectory="logs\"
      debuggingEnabled="false" />
  </system.webServer>
  <appSettings>
    <add key="PORT"                    value="3000" />
    <add key="NEXT_PUBLIC_API_URL"     value="http://localhost:5000" />
  </appSettings>
</configuration>
```

💡 **Alternative: Run Next.js as a Windows Service**

Instead of iisnode, you can run `node server.js` as a Windows Service using **NSSM** (Non-Sucking Service Manager). This is simpler and avoids iisnode complexity. Use IIS only as a reverse proxy in front of the Node.js process on port 3000.

Step 8 — IIS Reverse Proxy (ARR)

Use Application Request Routing (ARR) to expose the frontend on port 80/443 and proxy API requests to the gateway.


```
C:\inetpub\wwwroot (or dedicated site)\web.config — ARR reverse proxy
<?xml version="1.0" encoding="utf-8"?>
<configuration>
  <system.webServer>
    <rewrite>
      <rules>
        <!-- Proxy /api/* to the YARP Gateway on port 5000 -->
        <rule name="API Gateway Proxy" stopProcessing="true">
          <match url="^api/(.*)" />
          <action type="Rewrite"
                  url="http://localhost:5000/api/{R:1}" />
        </rule>
        <!-- Proxy all other traffic to Next.js frontend on port 3000 -->
        <rule name="Frontend Proxy" stopProcessing="true">
          <match url="(.*)" />
          <action type="Rewrite"
                  url="http://localhost:3000/{R:1}" />
        </rule>
      </rules>
    </rewrite>
    <security>
      <requestFiltering>
        <requestLimits maxAllowedContentLength="52428800" /> <!-- 50 MB -->
      </requestFiltering>
    </security>
  </system.webServer>
</configuration>
```

> ⚠️ **Enable ARR Proxy**
>
> After installing ARR, you must enable the proxy in IIS Manager: open **Application Request Routing Cache** at the server level → click **Server Proxy Settings** → check **Enable proxy** → click Apply.

Step 9 — SSL / TLS Configuration

Bind an SSL certificate to the main IIS site (port 443) and redirect HTTP to HTTPS.

PowerShell — bind SSL certificate

```powershell
# Import your certificate (.pfx) into the Windows Certificate Store
$cert = Import-PfxCertificate `
  -FilePath "C:\certs\retailerp.pfx" `
  -CertStoreLocation "Cert:\LocalMachine\My" `
  -Password (ConvertTo-SecureString "certPassword" -AsPlainText -Force)

# Bind HTTPS to port 443
New-WebBinding -Name "RetailERP-Main" `
  -Protocol "https" -Port 443 -HostHeader "yourdomain.com"

$sslBinding = Get-Item "IIS:\SslBindings\0.0.0.0!443" -ErrorAction SilentlyContinue
if (-not $sslBinding) {
  $cert | New-Item "IIS:\SslBindings\0.0.0.0!443"
}
```

web.config — HTTP to HTTPS redirect (add to main site)

```xml
<rewrite>
  <rules>
    <rule name="HTTP to HTTPS" stopProcessing="true">
      <match url="(.*)" />
      <conditions>
        <add input="{HTTPS}" pattern="off" ignoreCase="true" />
      </conditions>
      <action type="Redirect"
              url="https://{HTTP_HOST}/{R:1}"
              redirectType="Permanent" />
    </rule>
  </rules>
</rewrite>
```

Windows Service Alternative (no iisnode)

Each microservice and the Next.js frontend can also be run as native Windows Services using **NSSM** (Non-Sucking Service Manager), avoiding IIS hosting entirely for the backend.

PowerShell — register services with NSSM

```powershell
# Download nssm.exe from nssm.cc, place in C:\tools\nssm.exe

# Register Inventory service
C:\tools\nssm.exe install RetailERP-Inventory dotnet
C:\tools\nssm.exe set    RetailERP-Inventory AppParameters `
  "C:\RetailERP\services\Inventory\RetailERP.Inventory.API.dll"
C:\tools\nssm.exe set    RetailERP-Inventory AppDirectory `
  "C:\RetailERP\services\Inventory"
C:\tools\nssm.exe set    RetailERP-Inventory AppEnvironmentExtra `
  "ASPNETCORE_ENVIRONMENT=Production" `
  "ASPNETCORE_URLS=http://localhost:5003" `
  "ConnectionStrings__DefaultConnection=Server=localhost;..."
C:\tools\nssm.exe set    RetailERP-Inventory Start SERVICE_AUTO_START
Start-Service RetailERP-Inventory

# Register Next.js frontend
C:\tools\nssm.exe install RetailERP-Frontend node
C:\tools\nssm.exe set    RetailERP-Frontend AppParameters `
  "C:\RetailERP\frontend\server.js"
C:\tools\nssm.exe set    RetailERP-Frontend AppDirectory `
  "C:\RetailERP\frontend"
C:\tools\nssm.exe set    RetailERP-Frontend AppEnvironmentExtra `
  "PORT=3000" `
  "NEXT_PUBLIC_API_URL=http://localhost:5000"
C:\tools\nssm.exe set    RetailERP-Frontend Start SERVICE_AUTO_START
Start-Service RetailERP-Frontend
```

CHAPTER 4

Configuration Reference

appsettings.json, environment variables, and JWT configuration per deployment target

appsettings per Environment

ASP.NET Core loads configuration in order: `appsettings.json` → `appsettings.{Environment}.json` → environment variables. Environment variables always take highest precedence and use `__` as the section separator.

Setting Key	Development Value	Docker / Production Value
`ASPNETCORE_ENVIRONMENT`	`Development`	`Docker / Production`
`ConnectionStrings__DefaultConnection`	localhost, Trusted_Connection	sqlserver container / SQL Server host, SA auth
`ASPNETCORE_URLS`	`http://localhost:5001`	`http://+:8080`
`Jwt__Secret`	Dev secret (32+ chars)	Production secret from env var / secret store
`Jwt__ExpiryInHours`	`1`	`8 (recommended for production)`
`Cors__Origins__0`	`http://localhost:3000`	https://yourdomain.com
`Serilog__MinimumLevel__Default`	`Information`	`Warning`

Frontend Environment Variables

Variable	Description	Docker Value	IIS/Production Value
`NEXT_PUBLIC_API_URL`	Backend API gateway base URL	`http://localhost:5000`	`https://yourdomain.com`
`NEXT_PUBLIC_APP_NAME`	Application display name	`EL CURIO`	`EL CURIO`
`PORT`	Next.js server port (runtime only)	`3000`	`3000`
`NODE_ENV`	Node environment	`production`	`production`
`NEXT_TELEMETRY_DISABLED`	Disable Next.js telemetry	`1`	`1`

NEXT_PUBLIC_ Variables Are Baked In at Build Time

Variables prefixed with `NEXT_PUBLIC_` are embedded into the JavaScript bundle during `npm run build`. If you need to change the API URL after building, you must rebuild the frontend. For runtime-configurable URLs, use a server-side API route to fetch configuration.

JWT & Security Settings

Generating a Strong JWT Secret

bash / PowerShell

```bash
# Linux / macOS
openssl rand -base64 48

# PowerShell (Windows)
[Convert]::ToBase64String((New-Object byte[] 48 | ForEach-Object {
  [System.Security.Cryptography.RandomNumberGenerator]::GetBytes(1)[0]
}))

# Expected output: a 64-character base64 string, e.g.:
# k9QwZ3mP7xRt2vN6jH8eY4cL5aG1bF0dI3nO9uM7sX2pW...
```

Setting	Value	Notes
`Jwt__Secret`	Min 32 chars, 48+ recommended	Same value across all services
`Jwt__Issuer`	`RetailERP.Auth`	Must match exactly in all services
`Jwt__Audience`	`RetailERP.Client`	Must match exactly in all services
`Jwt__ExpiryInHours`	`1–24`	Shorter = more secure; users re-authenticate more often

JWT Secret Must Be Identical Across All Services

Every microservice validates JWT tokens using the same secret. If any service has a different `Jwt__Secret`, it will reject tokens issued by the Auth service, causing 401 Unauthorized errors. Use environment variables or a shared secret store to ensure all services share the exact same value.

CHAPTER 5

Troubleshooting

Common issues and their resolutions for Docker and IIS deployments

Docker Common Issues

Symptom	Cause	Resolution
Container exits immediately	Missing DLL, wrong SERVICE_NAME build arg, or startup crash	Run `docker compose logs <service>` for the error. Verify build args match the actual project name.
SQL Server stays "starting"	Insufficient memory (SQL Server needs 2 GB minimum)	Increase Docker Desktop memory limit to 8 GB or more.
401 on all API calls	JWT secret mismatch between auth and other services, or CORS misconfiguration	Verify `Jwt__Secret` is identical in all service containers. Check `Cors__Origins` in gateway config.
Frontend shows "Network Error"	`NEXT_PUBLIC_API_URL` points to wrong host	Rebuild frontend image with correct env var. In Docker, the browser calls `localhost:5000` directly; ensure gateway port is exposed.
db-init container keeps restarting	SQL Server not healthy yet when db-init starts	db-init depends on `service_healthy`; if SQL Server health check fails, increase `retries` or `start_period` in `docker-compose.yml`.
Port already in use	Another process holds the port	Run `netstat -ano | findstr :5001` (Windows) or `ss -tlnp | grep 5001` (Linux) to find and stop the conflicting process.
Images fail to build	NuGet restore failures or no internet access	Run `docker compose build --no-cache --progress=plain` for full output. Check proxy settings if on a corporate network.

IIS Common Issues

Symptom	Cause	Resolution
HTTP 500.19 — Config error	ANCM not installed or web.config syntax error	Reinstall the .NET 8 Hosting Bundle. Validate web.config XML syntax.
HTTP 502.5 — Process failure	The dotnet process crashed on startup	Enable `stdoutLogEnabled="true"` in web.config and check the `logs\stdout_*.log` file for the exception.
HTTP 500.30 — Start failure	Missing connection string, wrong DLL name, or missing .NET runtime	Verify the `arguments` value in web.config matches the actual DLL filename. Verify .NET 8 runtime is installed.
Application Pool stops	Rapid failure restarts (5 failures in 5 minutes triggers stop)	Open IIS Manager → Application Pools → set **Rapid Fail Protection** to disabled during debugging. Fix the underlying startup error first.
Frontend shows blank page	iisnode not installed or Node.js not found	Verify iisnode.dll is registered in IIS. Ensure Node.js path is in the system PATH variable. Restart IIS (`iisreset`).
ARR not proxying	Proxy not enabled in ARR server settings	IIS Manager → server root → Application Request Routing Cache → Server Proxy Settings → check **Enable proxy**.
CORS errors in browser	Frontend domain not in `Cors__Origins`	Add the production domain to `Cors__Origins` in gateway and auth service environment variables.

Database Connectivity Issues

Symptom	Cause	Resolution
Login failed for user 'sa'	SQL Server Authentication not enabled, or wrong password	In SQL Server Management Studio: Server Properties → Security → select "SQL Server and Windows Authentication mode". Restart SQL Server service.
Cannot connect to server	SQL Server TCP/IP protocol disabled	SQL Server Configuration Manager → SQL Server Network Configuration → Protocols for MSSQLSERVER → enable TCP/IP. Restart SQL Server service.
Database tables not found	db-init scripts did not run or partially failed	Run the SQL scripts manually from the `database/` folder in order: schemas → tables → indexes → stored-procedures → seed-data.
Connection timeout in Docker	Service started before SQL Server was ready	The health check ensures SQL Server is ready before services start. If issues persist, increase `start_period` for `sqlserver` in docker-compose.yml.

Pre-Deployment Checklist

- ☐ Changed SQL Server SA password from default `RetailERP@2024!`

- ☐ Generated a new JWT secret (minimum 48 characters, base64 encoded)
- ☐ Updated Grafana admin password from `admin`
- ☐ Changed the application admin password (`Admin@123`) after first login
- ☐ Set `ASPNETCORE_ENVIRONMENT=Production` on all services (disables Swagger)
- ☐ Verified all 7 microservices respond to `/health` endpoint
- ☐ Verified database initialization completed (all schemas and seed data)
- ☐ SSL certificate installed and HTTPS redirect configured
- ☐ CORS origins updated to the production domain
- ☐ Log rotation configured (Docker json-file driver or Windows Event Log)
- ☐ Firewall rules restrict direct access to microservice ports (5001–5007)
- ☐ Only ports 80 and 443 exposed publicly; all others internal only
- ☐ Backup schedule configured for SQL Server database
- ☐ Monitoring alerts configured in Grafana for service downtime

Deployment Support

For deployment assistance, configuration questions, or production-specific issues, contact the RetailERP support team at **support@elcurio.com**. Reference this document version **v2.1 — March 2026** when raising a ticket.